

DBMS – SPECIAL ASSESSMENT

1. PROBLEM STATEMENT

The NV-CES (National Virtual Certification Examination System) requires reliable management of examination-slot bookings and examination-result updates. These operations involve multiple users accessing and modifying the same database records at the same time.

In a simple read-then-write approach, two candidates could attempt to book the last available seat simultaneously. Both transactions may read the same available capacity and attempt to reserve the same seat, resulting in **overbooking and inconsistent seat counts**.

Similarly, two examiners may attempt to update the result of the same candidate concurrently. Without proper concurrency control, one examiner's update could overwrite another examiner's update, resulting in a **lost-update anomaly**.

Therefore, the system requires transaction management and concurrency-control mechanisms that maintain database consistency while allowing multiple sessions to operate safely. The proposed implementation uses transactions, row-level locking through SELECT ... FOR UPDATE, COMMIT, ROLLBACK, SAVEPOINT, version tracking, and result-update history. The assessment requirements specifically identify transaction management, ACID properties, COMMIT/ROLLBACK/SAVEPOINT and concurrency control as core DBMS concepts.

2. OBJECTIVE

The main objective of Module 2 is to implement a reliable transaction and concurrency-management mechanism for the NV-CES database.

The specific objectives are:

1. To implement database transactions for examination-slot booking.
2. To prevent overbooking when multiple users attempt to reserve the same slot.
3. To implement row-level locking using SELECT ... FOR UPDATE.
4. To demonstrate the use of START TRANSACTION, COMMIT, and ROLLBACK.

5. To implement transaction-safe examination-result updates.
6. To prevent lost updates when two examiners modify the same result concurrently.
7. To maintain version information for result modifications.
8. To maintain an audit history of result changes.
9. To demonstrate partial transaction rollback using SAVEPOINT.
10. To validate transaction correctness using multiple MySQL sessions and test cases.

The implementation follows the assignment requirement to test transaction correctness under simulated concurrent load.

3. REQUIREMENTS AND ENVIRONMENT USED

3.1 Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Keyboard and mouse
- Sufficient storage for MySQL and project files

3.2 Software Requirements

Requirement	Used
Operating System	Windows
Database Management System	MySQL 8.0
Database	nvces_db
Interface	MySQL 8.0 Command Line Client
Programming/Query Language	SQL
Development Environment	MySQL Command Line Client
Concurrency Testing	Two independent MySQL sessions

The assignment permits MySQL/MySQL Workbench and specifically recommends multiple concurrent SQL sessions for simulating simultaneous booking and result-update requests.

3.3 Database Components Used

The major database components relevant to Module 2 are:

- exam_slots
- bookings
- results
- examiners
- result_update_history

The result_update_history table records the old score, new score, version number, examiner and update time, allowing the system to preserve the sequence of result modifications.

4. DESIGN / PROPOSED SOLUTION

4.1 Overview

The proposed solution uses **pessimistic concurrency control**. Before modifying a shared record, the transaction locks the required row using:

```
SELECT ... FOR UPDATE;
```

The lock ensures that another transaction attempting to modify the same row must wait until the first transaction completes.

The transaction follows the general sequence:

**Request → Start Transaction → Lock Required Row → Validate Data → Perform Update
→ Record Changes → COMMIT**

If an error occurs, the transaction can be cancelled using:

```
ROLLBACK;
```

This prevents partial changes from being permanently stored.

4.2 Booking Transaction Design

The booking process locks the corresponding row in exam_slots before checking availability.

Booking workflow

1. Start a transaction.
2. Select the required slot using FOR UPDATE.
3. Lock the slot row.
4. Check whether booked_count < capacity.
5. Insert the booking.
6. Increment booked_count.
7. Commit the transaction.
8. If the operation fails, rollback the transaction.

This prevents two concurrent sessions from independently seeing the same remaining seat.

4.3 Result Transaction Design

The result-update process follows a similar transaction-based approach.

1. Start a transaction.
2. Locate the result using attempt_id.
3. Lock the result row using FOR UPDATE.
4. Read the current score and version.
5. Update the score.
6. Increment the version number.
7. Insert the change into result_update_history.
8. Commit the transaction.

The enhanced implementation records both examiner decisions in the history table while the results table stores the latest committed score.

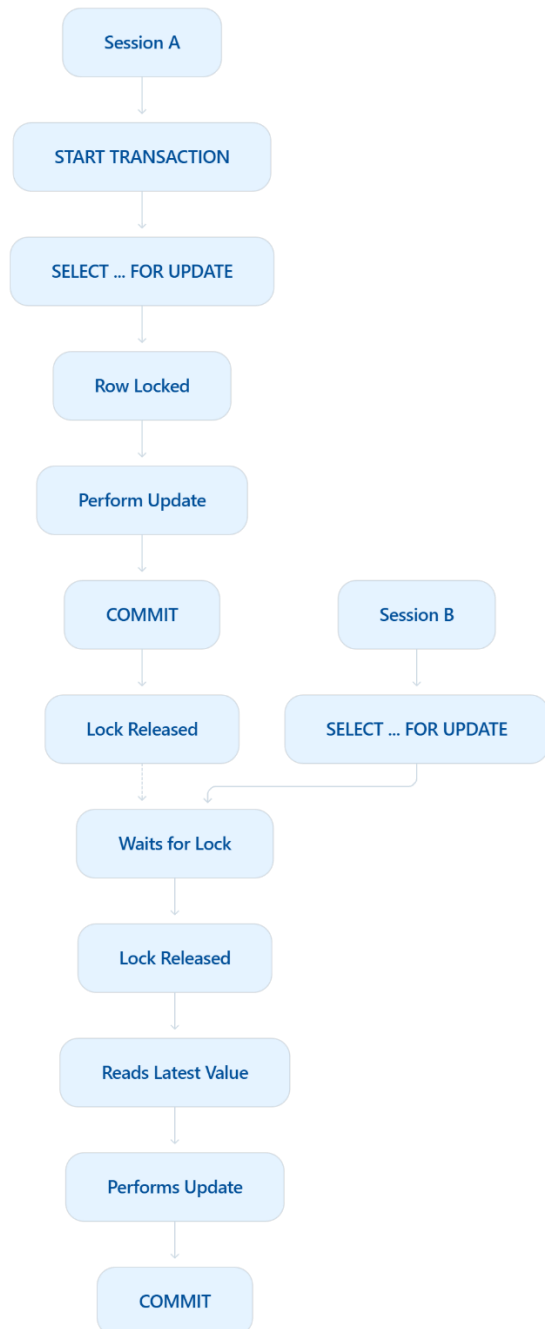
4.4 Concurrency-Control Design

Two independent MySQL sessions are used:

Session A → Transaction 1

Session B → Transaction 2

When both transactions request the same locked row:



The project documentation specifies this two-terminal approach for demonstrating both booking and result concurrency.

4.5 Result Version Tracking

Each result contains a version number.

Example:

Initial:

Score = 55.00

Version = 0

Examiner 1:

Score = 58.00

Version = 1

Examiner 2:

Score = 60.00

Version = 2

The live results table therefore contains the latest committed value, while `result_update_history` preserves the individual update events.

4.6 Transaction Properties

The implementation supports the major transaction properties:

Atomicity

A transaction is completed completely or rolled back completely.

Consistency

Constraints such as slot capacity and valid result relationships are maintained.

Isolation

Concurrent transactions operating on the same row are serialized using row-level locking.

Durability

Once a transaction is committed, its changes remain stored in the database.

5. ALGORITHM / PSEUDOCODE / FLOWCHART

5.1 Algorithm for Concurrent Slot Booking

Algorithm: Transaction-Safe Slot Booking

1. Start transaction.
2. Lock the selected examination slot.
3. Read capacity and booked_count.
4. If booked_count $\geq$ capacity:
 - Rollback.
 - Reject booking.
5. Otherwise:
 - Insert booking record.
 - Increment booked_count.
 - Commit transaction.
6. End.

Pseudocode

BEGIN TRANSACTION

LOCK selected exam slot

READ capacity and booked_count

```
IF booked_count >= capacity THEN
    ROLLBACK
    Reject booking
ELSE
    INSERT booking
    INCREMENT booked_count
    COMMIT
    Confirm booking
END IF
```

5.2 Algorithm for Concurrent Result Update

1. Start transaction.
2. Lock the result row.
3. Read current score and version.
4. Store current score as old_score.
5. Update score with new value.
6. Increment version.
7. Insert an audit-history record.
8. Commit transaction.
9. End.

Pseudocode

```
BEGIN TRANSACTION
LOCK result row using FOR UPDATE
READ current score and version
```

old_score = current score

new_version = current version + 1

UPDATE result

SET final_score = new_score

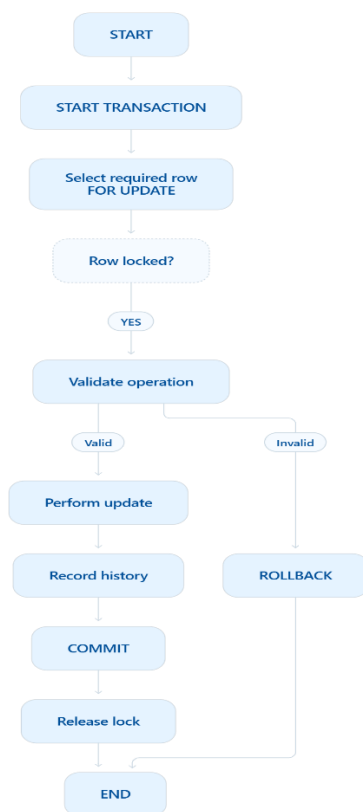
SET version = new_version

INSERT old_score, new_score,
version and examiner information
into result_update_history

COMMIT

END

5.3 Transaction Flowchart



6. IMPLEMENTATION / SOURCE CODE

The Module 2 implementation is distributed across the transaction and concurrency SQL scripts.

6.1 Booking Transaction

The booking transaction uses:

```
START TRANSACTION;
```

```
SELECT slot_id, capacity, booked_count  
FROM exam_slots  
WHERE slot_id = 2  
FOR UPDATE;
```

After verifying availability:

```
INSERT INTO bookings  
(slot_id, candidate_id, status)  
VALUES  
(2, 4, 'CONFIRMED');
```

```
UPDATE exam_slots  
SET booked_count = booked_count + 1  
WHERE slot_id = 2;
```

```
COMMIT;
```

If the operation cannot be completed:

```
ROLLBACK;
```

6.2 Result Transaction

The result transaction locks the result before modification:

```
START TRANSACTION;
```

```
SELECT result_id,  
       attempt_id,  
       final_score,  
       version  
FROM results  
WHERE attempt_id = 6  
FOR UPDATE;
```

The result is then updated and its version incremented.

```
UPDATE results  
SET final_score = 58.00,  
    version = version + 1,  
    status = 'PUBLISHED'  
WHERE attempt_id = 6;
```

The corresponding historical change is recorded in:

result_update_history

and the transaction is completed using:

```
COMMIT;
```

The implemented procedure was enhanced to lock the result row, obtain the previous score/version, update the live result and insert an audit record.

6.3 Savepoint

Savepoints are used when only part of a transaction needs to be undone.

General implementation:

```
START TRANSACTION;
```

```
-- Operation 1
```

```
SAVEPOINT before_second_operation;
```

```
-- Operation 2
```

```
ROLLBACK TO SAVEPOINT before_second_operation;
```

```
COMMIT;
```

This allows the transaction to undo changes after the savepoint without discarding the entire transaction.

6.4 Concurrency Demonstration

The concurrency demonstration uses:

- 09_concurrency_session1.sql
- 10_concurrency_session2.sql

Two MySQL command-line sessions are opened simultaneously. Session 2 attempts to access a row already locked by Session 1 and waits until Session 1 commits. The project documentation defines this procedure for both booking and result-update demonstrations.

7. TEST CASES AND EXPECTED/ACTUAL RESULTS

The following test cases are used to validate Module 2.

TC No.	Test Case	Expected Result	Actual Result	Status
TC01	Start booking transaction	Transaction starts successfully	Transaction started successfully	Pass
TC02	Lock an examination slot	Selected slot is locked	Slot row locked successfully	Pass
TC03	Book available slot	Booking should be accepted	Booking accepted	Pass
TC04	Commit successful booking	Booking and count update persist	Changes committed successfully	Pass
TC05	Attempt concurrent booking	Second session waits for locked row	Second session waits	Pass
TC06	Check slot after first commit	Latest booked_count should be visible	Updated count observed	Pass
TC07	Update examination result	Score should be updated	Score updated successfully	Pass
TC08	Concurrent result update	Second examiner waits for lock	Second session waited and then proceeded	Pass
TC09	Version tracking	Version should increase after update	Version increased from 0 to 1 to 2	Pass
TC10	Result history	Each committed result change should be recorded	Two history records generated	Pass
TC11	Rollback transaction	Uncommitted changes should be removed	Rollback successful	Pass
TC12	Savepoint rollback	Only changes after savepoint should be undone	Partial rollback successful	Pass
TC13	Final result verification	Latest committed result should be displayed	Final score displayed as 60.00	Pass

TC No.	Test Case	Expected Result	Actual Result	Status
TC14	Slot capacity verification	booked_count must not exceed capacity	Count remained within capacity	Pass
TC15	Concurrent transaction consistency	No inconsistent final state	Consistent final state obtained	Pass

The assignment's validation requirements specifically include concurrent last-seat booking, concurrent result updates and rollback testing.

7.1 Booking Concurrency Test

The slot was configured with:

Capacity = 4

Booked Count = 3

Therefore, one seat was available.

Session 1 acquired the row lock and performed the booking.

After the booking:

Capacity = 4

Booked Count = 4

Session 2 subsequently observed the updated state after Session 1 released the lock.

7.2 Result Concurrency Test

Initial state:

Attempt ID = 6

Final Score = 55.00

Version = 0

First examiner transaction:

55.00 → 58.00

Version 0 → 1

Second examiner transaction:

58.00 → 60.00

Version 1 → 2

Final state:

Final Score = 60.00

Version = 2

Status = PUBLISHED

The history table stores both transitions:

55.00 → 58.00

58.00 → 60.00

This demonstrates that the live result contains the latest value while the audit table preserves the sequence of committed updates.

8. EXECUTION SCREENSHOTS / OUTPUT

Screenshot 1 – Successful Booking Transaction

What it demonstrates:

Successful booking transaction, update of the slot count and transaction completion.

```
MySQL 8.0 Command Line Client

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT slot_id, capacity, booked_count
-> FROM exam_slots
-> WHERE slot_id = 2
-> FOR UPDATE;

+-----+-----+-----+
| slot_id | capacity | booked_count |
+-----+-----+-----+
| 2       | 4       | 3           |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> INSERT INTO bookings
-> (slot_id, candidate_id, status)
-> VALUES (2, 4, 'CONFIRMED');
Query OK, 1 row affected (0.00 sec)

mysql> UPDATE exam_slots
-> SET booked_count = booked_count + 1
-> WHERE slot_id = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT slot_id, capacity, booked_count
-> FROM exam_slots
-> WHERE slot_id = 2
-> FOR UPDATE;

+-----+-----+-----+
| slot_id | capacity | booked_count |
+-----+-----+-----+
| 2       | 4       | 4           |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Figure 1: Successful examination-slot booking transaction and COMMIT operation.

Screenshot 2 – Initial Result State

What it demonstrates:

Initial result state before concurrent result updates:

Attempt ID = 6

Final Score = 55.00

Version = 0


```
MySQL 8.0 Command Line Client
Rows matched: 1  Changed: 1  Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT slot_id, capacity, booked_count
-> FROM exam_slots
-> WHERE slot_id = 2
-> FOR UPDATE;
+-----+-----+-----+
| slot_id | capacity | booked_count |
+-----+-----+-----+
|      2 |      4 |           4 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT result_id, attempt_id, final_score, version
-> INTO @res_id, @att_id, @old_score, @cur_ver
-> FROM results
-> WHERE attempt_id = 6
-> FOR UPDATE;
Query OK, 1 row affected (0.00 sec)

mysql> SELECT @res_id AS result_id,
->             @att_id AS attempt_id,
->             @old_score AS old_score,
->             @cur_ver AS current_version;
+-----+-----+-----+-----+
| result_id | attempt_id | old_score | current_version |
+-----+-----+-----+-----+
|         5 |         6 |    55.00 |              0 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Figure 2: Initial result state before concurrent examiner updates.

Screenshot 3 – Second Session After First Update

What it demonstrates:

Session 2 obtains the row after Session 1 commits and observes:

Final Score = 58.00

Version = 1

```
MySQL 8.0 Command Line Client
+-----+
| DATABASE() |
+-----+
| nvces_db |
+-----+
1 row in set (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT slot_id, capacity, booked_count
-> FROM exam_slots
-> WHERE slot_id = 2
-> FOR UPDATE;
ERROR 1205 (HY000): Lock wait timeout exceeded; try restarting transaction
mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql> USE nvces_db;
Database changed
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT result_id, attempt_id, final_score, version
-> INTO @s2_res_id, @s2_att_id, @s2_old_score, @s2_cur_ver
-> FROM results
-> WHERE attempt_id = 6
-> FOR UPDATE;
Query OK, 1 row affected (48.02 sec)

mysql> SELECT @s2_res_id AS result_id,
-> @s2_att_id AS attempt_id,
-> @s2_old_score AS unblocked_score,
-> @s2_cur_ver AS unblocked_version;
+-----+-----+-----+-----+
| result_id | attempt_id | unblocked_score | unblocked_version |
+-----+-----+-----+-----+
| 5 | 6 | 58.00 | 1 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

Figure 3: Second concurrent session observing the committed result update.

Screenshot 4 – Final Result and Update History

What it demonstrates:

Final result:

Final Score = 60.00

Version = 2

Status = PUBLISHED

and the two historical updates:

55.00 → 58.00

58.00 → 60.00

```
MySQL 8.0 Command Line Client
-> SET final_score = 60.00,
-> status = 'PUBLISHED',
-> version = @s2_cur_ver + 1,
-> published_at = CURRENT_TIMESTAMP
-> WHERE attempt_id = 6;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> INSERT INTO result_update_history
-> (result_id, examiner_id, old_score, new_score, version_no, updated_at)
-> VALUES
-> (@s2_res_id, 4, @s2_old_score, 60.00,
-> @s2_cur_ver + 1, CURRENT_TIMESTAMP);
Query OK, 1 row affected (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT attempt_id, final_score, status, version
-> FROM results
-> WHERE attempt_id = 6;
+-----+-----+-----+-----+
| attempt_id | final_score | status | version |
+-----+-----+-----+-----+
| 6 | 60.00 | PUBLISHED | 2 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT history_id, result_id, examiner_id,
-> old_score, new_score, version_no, updated_at
-> FROM result_update_history
-> WHERE result_id = 5
-> ORDER BY version_no;
+-----+-----+-----+-----+-----+-----+-----+
| history_id | result_id | examiner_id | old_score | new_score | version_no | updated_at |
+-----+-----+-----+-----+-----+-----+-----+
| 4 | 5 | 3 | 55.00 | 58.00 | 1 | 2026-09-01 14:41:55 |
| 5 | 5 | 4 | 58.00 | 60.00 | 2 | 2026-09-01 14:43:05 |
+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
```

Figure 4: Final result state with version 2 and recorded result-update history.

9. ANALYSIS AND DISCUSSION

9.1 Transaction Management

Transactions ensure that related database operations are treated as a single logical unit. For example, a booking requires both insertion into bookings and incrementing booked_count. Performing only one of these operations would create an inconsistent database state.

Using COMMIT ensures that all successful operations become permanent, while ROLLBACK removes uncommitted changes.

9.2 Effectiveness of Row-Level Locking

SELECT ... FOR UPDATE provides pessimistic locking by locking the selected database row before modification.

In the booking test, Session 1 locked the slot row while Session 2 attempted to access the same row. Session 2 had to wait until Session 1 completed its transaction.

This ensures that both sessions do not simultaneously use the same remaining seat.

The project execution procedure explicitly demonstrates Session B waiting on the locked slot and continuing after Session A commits.

9.3 Result Concurrency

The result-update experiment demonstrates controlled concurrent access to the same result.

The initial result was:

55.00, Version 0

The first transaction changed it to:

58.00, Version 1

The second transaction then observed the committed value and changed it to:

60.00, Version 2

Therefore, the second examiner did not update an outdated version of the record.

The implementation accurately treats 60.00 as the final live score while retaining the two examiner decisions in `result_update_history`.

9.4 Audit History

The `result_update_history` table provides an additional layer of accountability.

It records:

- Previous score

- New score
- Version number
- Examiner
- Update timestamp

This makes it possible to determine how the final result was reached without incorrectly storing multiple conflicting live results.

9.5 Savepoints

Savepoints provide finer transaction control. Instead of rolling back an entire transaction, the system can undo operations performed after a particular savepoint.

This is useful when a transaction contains multiple stages and only one stage needs correction.

9.6 Advantages

The proposed transaction and concurrency mechanism provides:

1. Prevention of slot overbooking.
2. Protection against inconsistent concurrent updates.
3. Atomic booking and result operations.
4. Controlled rollback.
5. Partial rollback using savepoints.
6. Result version tracking.
7. Historical auditability.
8. Better consistency during concurrent access.

The assignment identifies elimination of double-booked slots and lost result updates as key expected benefits of the solution.

9.7 Limitations

The solution also has some limitations:

- Row locking can cause transactions to wait when many users access the same resource.
- Long-running transactions can increase lock contention.
- Maintaining result history requires additional storage.
- The concurrency tests simulate concurrent users using multiple SQL sessions rather than production-scale traffic.
- Stronger consistency can reduce throughput under extremely high concurrent load.

These trade-offs are consistent with the project requirements, which identify locking overhead and scalability as considerations.

10. CONCLUSION

Module 2 successfully implements transaction and concurrency management for the NV-CES examination system.

The booking transaction uses row-level locking to ensure that simultaneous users cannot incorrectly reserve the same remaining seat. The result-management transaction similarly locks the result record before modification, preventing stale concurrent updates.

The practical concurrency experiment demonstrated that one session waits while another session holds the required lock. After the first transaction commits, the second session reads the latest committed state and performs its update.

The result-update demonstration progressed from **55.00** → **58.00** → **60.00**, with the version changing from **0** → **1** → **2**. The `result_update_history` table preserved both committed update events while the results table retained the final score of **60.00**.

Overall, the implementation demonstrates the practical application of **ACID transactions, row-level locking, COMMIT, ROLLBACK, SAVEPOINT, concurrency control, version tracking and audit history** in a relational database system.

11. INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS

S. No.	Module	Individual Contribution
1	Module 2 – Transaction & Concurrency Management	Made the primary contribution to transaction and concurrency implementation. Implemented booking and result transactions; used START TRANSACTION, COMMIT, ROLLBACK, and SAVEPOINT; implemented row-level locking using SELECT ... FOR UPDATE; performed concurrent booking and result-update experiments using multiple MySQL sessions; implemented version tracking and result-update history; analysed concurrency behaviour and validated the corresponding test cases and execution results.
2	Module 1 – Database Design & Query Optimization	Contributed to database schema design, table creation, relationships, sample data, indexing, hashing and collision handling, SQL query development, query optimization and execution-plan analysis. Also supported project integration, testing and documentation.
3	Module 3 - Security, Access Control & Performance Evaluation	Users, privileges, roles, access control, performance benchmarking, validation

Individual Contribution

As the primary contributor to Module 2, I was responsible for implementing and validating the transaction and concurrency-control mechanisms. Particular emphasis was placed on the practical demonstration of concurrent slot booking and concurrent result updates using separate MySQL sessions.

12. REFERENCES

1. MySQL 8.0 Documentation – Transactional and locking concepts.
2. MySQL 8.0 Documentation – START TRANSACTION, COMMIT, ROLLBACK and SAVEPOINT.
3. MySQL 8.0 Documentation – SELECT ... FOR UPDATE and InnoDB locking.
4. Database Management Systems course material – Transaction Management and Concurrency Control.
5. NV-CES project SQL scripts and project documentation.

Project scripts referenced:

07_booking_transaction.sql
08_result_transaction.sql
09_concurrency_session1.sql
10_concurrency_session2.sql
11_savepoints.sql
13_test_cases.sql

The project documentation identifies these scripts as the transaction, result-processing, concurrency and savepoint components of the implementation.

13. ONE-PAGE WRITE-UP

NV-CES: Transaction and Concurrency Management

Introduction

NV-CES is a database-based examination management system designed to handle examination slots, candidate bookings and examination results. A major challenge in such a system is ensuring data consistency when multiple users access the same database records simultaneously. Module 2 addresses this problem through transaction management and concurrency control.

Problem

Without transaction control, two candidates may attempt to book the last available examination seat at the same time. Both transactions could read the same available capacity and create conflicting booking records. Similarly, two examiners updating the same candidate's result could overwrite one another's changes. These situations can lead to overbooking, incorrect result data and loss of important updates.

Proposed Solution

The system uses **pessimistic concurrency control** with MySQL transactions and row-level locking. The SELECT ... FOR UPDATE statement locks the required row before modification. The transaction then performs the required operations and completes using COMMIT. If an error occurs, ROLLBACK is used to remove the uncommitted changes.

For slot booking, the system first locks the selected row in exam_slots and checks whether seats are available. If capacity is available, the booking is inserted and booked_count is incremented before committing the transaction.

For result processing, the system locks the result record before updating it. The score and version number are read, the new score is stored, the version is incremented and the change is inserted into result_update_history.

Concurrency Demonstration

The implementation was tested using two independent MySQL sessions. During the booking experiment, one session locked an examination slot while the second session attempted to access the same row. The second session waited until the first transaction committed. It then observed the updated booking count.

The result concurrency experiment started with:

Score = 55.00

Version = 0

The first examiner updated the score to:

58.00

Version = 1

After the first transaction committed, the second examiner obtained the lock and observed the updated score. The second transaction then changed the score to:

60.00

Version = 2

The final live result was therefore **60.00**, while the history table preserved both changes: **55.00** → **58.00** and **58.00** → **60.00**.

Results

The implementation successfully demonstrated transaction atomicity, row-level locking, controlled concurrent access, rollback and version tracking. The booking test ensured that the slot count remained within its capacity. The result test ensured that concurrent examiner updates were serialized instead of operating on stale data.

Conclusion

Module 2 demonstrates how DBMS transaction and concurrency-control concepts can be applied to a practical examination system. The use of transactions, SELECT ... FOR UPDATE, COMMIT, ROLLBACK, SAVEPOINT, version tracking and audit history provides a reliable mechanism for maintaining consistency and accountability. The implementation successfully addresses the major concurrency problems associated with examination-slot booking and result processing.

